Phonon Dispersion in a One-Dimensional Multi-atomic Lattice

Guhan Dhanasekharan (EP24BTECH11011), Kumud Jalori (EP24BTECH11015), Naman Verma (EP24BTECH11016), Nazakar Abhilash (EP24BTECH11017), Nithin Surya S (EP24BTECH11018), Panduru Sri Abish (EP24BTECH11019), Prasang Das (EP24BTECH11020), Praveen Kumar (EP24BTECH11021), S Haresh (EP24BTECH11022), and S Kshiteesh (EP24BTECH11023)
Department of Engineering Physics
(Dated: November 11, 2026)

We compute the phonon dispersion of a one-dimensional lattice carrying p atoms of arbitrary mass per unit cell, coupled by nearest-neighbour springs. The equations of motion are reduced by a Bloch ansatz to a $p \times p$ Hermitian eigenvalue problem, which we diagonalise numerically across the first Brillouin zone. The method is validated against the closed-form monatomic and diatomic solutions, reproducing them to 4×10^{-14}. An interactive interface built with Streamlit exposes p, the masses, the spring constant and the lattice constant, and reports the acoustic and optical branches, their frequency limits, the bandgaps between them, and the eigenfrequencies at the zone centre and edge. Three parameter sets demonstrate that the gap is controlled by mass contrast and closes entirely when the masses are equal, while the spring constant sets the overall frequency scale as $\sqrt{K}$ without altering the shape of the spectrum.

I. INTRODUCTION

A crystal at finite temperature is not a static array of atoms: each atom oscillates about its equilibrium site, and because the atoms are coupled, those oscillations propagate as waves. The quantised normal modes of these waves are phonons, and the relation $\omega(q)$ between a mode's frequency and its wavevector — the dispersion relation — governs the solid's heat capacity, thermal conductivity and sound velocity.

The one-dimensional chain is the standard setting for this problem because it is exactly solvable and still contains the essential physics. A chain with one atom per cell supports a single *acoustic* branch. Placing p atoms in the cell produces p branches: one acoustic and $p-1$ *optical*. Where the masses differ, a frequency gap opens between them, inside which no travelling wave exists at any wavevector.

This report derives the general p-atom dispersion relation, implements it, validates it against the cases that can be solved by hand, and uses it to examine how masses and spring constants control the spectrum.

II. THEORY

A. Equations of motion

Consider a chain of identical unit cells of length a, as in Fig. 1(a). Cell n contains p atoms of masses $m_1, \ldots, m_p$. Let u_j^n be the displacement of atom j in cell n from its equilibrium position. Nearest neighbours are joined by springs of constant K, so in the harmonic approximation the force on an atom is linear in the *relative* displacements of its two neighbours:

$$m_j \ddot{u}_j^n = K\left(u_{j+1}^n - u_j^n\right) + K\left(u_{j-1}^n - u_j^n\right). \qquad (1)$$

The chain is closed by the identifications $u_{p+1}^n \equiv u_1^{n+1}$ and $u_0^n \equiv u_p^{n-1}$: the last atom of a cell bonds to the first atom of the next. Equation (1) is a coupled system of infinitely many ordinary differential equations, one per atom.

B. Bloch ansatz and the dynamical matrix

The system is invariant under translation by a, so by Bloch's theorem the solutions are plane waves modulated by a cell-periodic amplitude. We take

$$u_j^n = \frac{1}{\sqrt{m_j}}\, e_j\, e^{i(qna-\omega t)}, \qquad (2)$$

where the factor $m_j^{-1/2}$ is a choice, not a restriction: it makes the resulting matrix Hermitian rather than merely similar to a Hermitian matrix, which is what allows a stable eigensolver to be used below.

Substituting (2) into (1) and cancelling the common exponential turns the infinite system into a finite $p \times p$ eigenvalue problem,

$$D(q)\,\mathbf{e} = \omega^2 \mathbf{e}, \qquad (3)$$

with the *mass-weighted dynamical matrix*

$$D_{jj} = \frac{2K}{m_j}, \qquad (4)$$

$$D_{j,j+1} = D_{j+1,j} = -\frac{K}{\sqrt{m_j m_{j+1}}}, \qquad j = 1, \ldots, p-1, \qquad (5)$$

$$D_{1,p} = -\frac{K\, e^{-iqa}}{\sqrt{m_1 m_p}}, \qquad D_{p,1} = D_{1,p}^{*}. \qquad (6)$$

Only the inter-cell element (6) depends on q; the phase e^{-iqa} is the Bloch factor picked up on crossing a cell

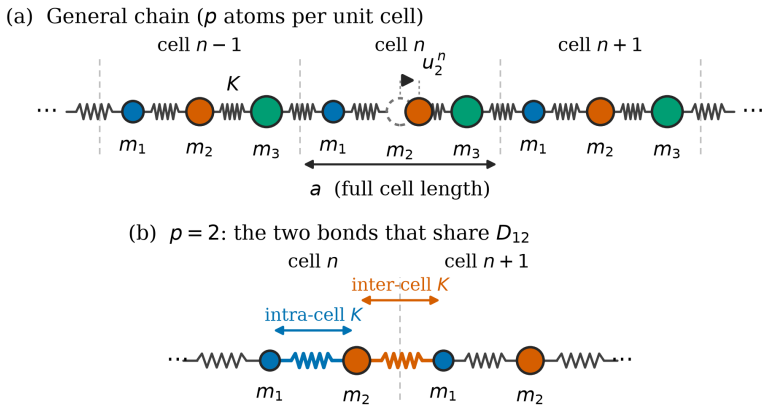


FIG. 1. The model. (a) Three unit cells of the general p-atom chain; circle area scales with mass, a is the full cell length, and u_2^n marks a longitudinal displacement from the equilibrium site (dashed). (b) The $p = 2$ case. Both bonds join the same pair m_1–m_2, one inside cell n and one across the boundary, so both land in the same matrix element D_{12} and the implementation must *accumulate* their contributions rather than overwrite one with the other — see Sec. II.

boundary. Since $D(q)^\dagger = D(q)$, its eigenvalues ω^2 are real, and since the harmonic lattice is mechanically stable they are non-negative.

a. A caution on conventions. Here a is the *full unit-cell length*. Many texts treating the diatomic chain set a to the nearest-neighbour spacing, making the cell $2a$ long; their $\cos(qa)$ is our $\cos(qa/2)$. Mixing the two conventions introduces a silent factor of two into every plotted frequency. All results below use the full-cell convention.

b. A caution on the $p = 2$ case. When the cell holds exactly two atoms, the intra-cell bond (5) and the inter-cell bond (6) occupy the *same* matrix element, as Fig. 1(b) shows: atom 1 bonds to atom 2 twice, once inside the cell and once across the boundary. The two contributions must therefore be added, giving $D_{12} = -K\left(1 + e^{-iqa}\right)/\sqrt{m_1 m_2}$. Overwriting rather than accumulating is an easy implementation error, and it breaks the physics: the acoustic branch no longer vanishes at $q = 0$, violating translational invariance.

C. Branch structure

Diagonalising (3) at each q in the first Brillouin zone, $q \in [-\pi/a, \pi/a]$, gives p frequencies. Ordering them ascending at every q:

- the lowest, $\omega_1(q)$, is the **acoustic** branch. It satisfies $\omega_1(0) = 0$, because a uniform translation of the entire crystal costs no energy — this is a direct consequence of $D(0)$ having a null vector $e_j \propto \sqrt{m_j}$. Near $q = 0$ it is linear, $\omega_1 \simeq v_s|q|$, with

$$v_s = a\sqrt{\frac{K}{p\sum_j m_j}}, \tag{7}$$

obtained by treating the cell as p springs in series over a length a carrying total mass $\sum_j m_j$. This is the speed of sound in the chain.

- the remaining $p - 1$ branches are **optical**. They stay finite as $q \to 0$, where they describe the atoms within a cell oscillating against one another with the cell's centre of mass at rest.

D. Closed forms used for validation

For $p = 1$ (mass m) the matrix is 1×1 and

$$\omega(q) = 2\sqrt{\frac{K}{m}}\left|\sin\frac{qa}{2}\right|. \tag{8}$$

For $p = 2$ (masses m_1, m_2) the 2×2 determinant gives, writing $\mu^{-1} \equiv m_1^{-1} + m_2^{-1}$ for the reduced mass,

$$\omega_\pm^2 = \frac{K}{\mu} \pm K\left[\frac{1}{\mu^2} - \frac{4\sin^2(qa/2)}{m_1 m_2}\right]^{1/2}, \tag{9}$$

with ω_- acoustic and ω_+ optical. At the zone boundary these reduce to $\omega_- = \sqrt{2K/m_>}$ and $\omega_+ = \sqrt{2K/m_<}$, where $m_>$ and $m_<$ are the larger and smaller mass — so the gap at the zone edge is set purely by the mass contrast and vanishes when $m_1 = m_2$.

III. NUMERICAL METHOD

The procedure is deliberately direct; no iteration or convergence criterion is involved.

1. Build a grid of n_q wavevectors spanning the first Brillouin zone. n_q is kept odd so that the grid contains $q = 0$ as well as $\pm\pi/a$: in a nearest-neighbour chain the branch extrema, and hence the gap edges, sit at exactly those points.

2. At each q, assemble $D(q)$ from (4)–(6) as a complex $p \times p$ array, accumulating the inter-cell bond as noted above.

3. Diagonalise with `numpy.linalg.eigvalsh`, the solver for Hermitian matrices. It returns real eigenvalues in ascending order, which is exactly the branch ordering we want, so no sorting step is needed.

4. Take $\omega = \sqrt{\omega^2}$, clipping values at zero first: at $q = 0$ the acoustic eigenvalue is zero up to rounding and can emerge as a small negative number, whose square root would be `nan`. Only round-off-sized negatives, below $100\,\varepsilon\,\|D\|$, are clipped; anything more negative means a mechanically unstable lattice (for instance $K < 0$) and is reported as an error rather than silently returned as $\omega = 0$.

5. Extract the per-branch minima and maxima, and identify a gap between branches i and $i+1$ whenever $\min_q \omega_{i+1} - \max_q \omega_i$ exceeds $10^{-9}\,\omega_{\max}$. The threshold is relative so that the result does not depend on the units of K and m.

The cost is n_q diagonalisations of a $p \times p$ matrix, negligible for the sizes here, which is why the interface can recompute the full spectrum on every slider movement.

The whole of the physics is the assembly in step 2 together with the single call in step 3; the rest of the codebase computes derived quantities, validates them, or draws them. Listing 1 is the implementation, reproduced verbatim from `phonon.py`.

Listing 1. Construction of the dynamical matrix for $p \geq 2$. `K` holds one spring constant per bond, `K[j]` joining atom j to $j+1$ and `K[-1]` the inter-cell bond; a scalar K is expanded to equal values, the case used throughout this report. The `+=` on the inter-cell bond is the subtlety discussed in Sec. II.

```
sm = np.sqrt(m)      # sqrt(m_j)*sqrt(m_k): cannot overflow
D = np.zeros((p, p), dtype=complex)
# each atom sits between the bond on its left and on its right
np.fill_diagonal(D, (np.roll(K, 1) + K) / m)

# intra-cell bonds
for j in range(p - 1):
    c = -K[j] / (sm[j] * sm[j + 1])
    D[j, j + 1] = c
    D[j + 1, j] = c

# inter-cell bond, carries all the q-dependence
c = -K[-1] * np.exp(-1j * q * a) / (sm[0] * sm[-1])
D[0, -1] += c
D[-1, 0] += np.conj(c)

# D is Hermitian, so eigvalsh gives real, ascending eigenvalues
w2 = np.linalg.eigvalsh(D)
omega = np.sqrt(np.clip(w2, 0.0, None))
```

A. Validation

Seven checks run in `test_phonon.py`; all pass.

Figure 2 shows the diatomic check graphically: the numerically obtained points lie on the analytic curves everywhere in the zone.

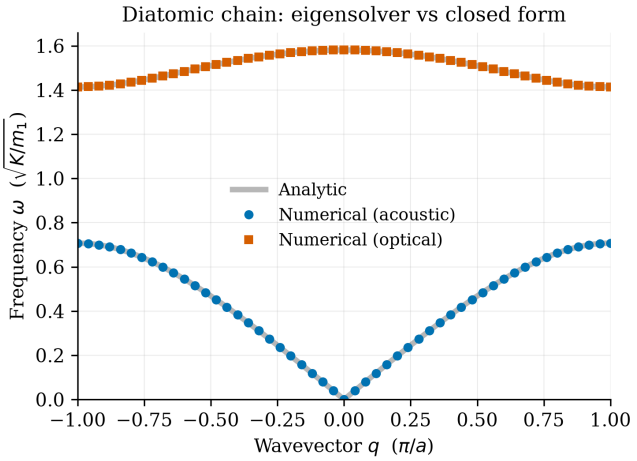


FIG. 2. Numerical eigenfrequencies (markers) against the closed-form diatomic solution Eq. (9) (grey), for $m = (1, 4)$, $K = 1$.

IV. THE INTERACTIVE INTERFACE

The interface is written with Streamlit and launched with `streamlit run app.py`. The sidebar accepts the number of atoms per cell p, one mass field per atom, the spring constant K, the lattice constant a, and the q-grid density; three presets load the demonstration cases below. Every change triggers a recomputation.

The main panel shows the dispersion with the acoustic branch drawn separately from the optical ones, each gap shaded across the zone and annotated with its width, and tables giving the frequency limits of every branch, every bandgap, the analytic sound velocity, and the eigenfrequencies at $q = 0$ and $q = \pi/a$. The density of states is plotted underneath. Frequencies are shown in the units implied by the inputs, $\omega \propto \sqrt{K/m}$, and equal units of $\sqrt{K/m_1}$ only when $K = m_1 = 1$, as in every result below.

Two situations are flagged rather than plotted silently. When every mass is equal the true period is a/p, so the upper branches are the acoustic branch folded back into the smaller zone; they are labelled "folded acoustic" rather than optical. When the selected branch is degenerate with another at the chosen q, the mode-displacement panel warns that the drawn pattern is one arbitrary member of a degenerate pair. Inputs are bounded, and anything that would make $D(q)$ non-finite or give a negative ω^2 produces an error message rather than a crash.

Figures 3 and 4 show the interface running on two of the demonstration cases. Switching preset from the diatomic Case A to the triatomic Case C changes the number of mass fields, adds a third branch to the plot, and produces a second bandgap row in the table — all from the same code path, with no special-casing for the number of atoms per cell. The values displayed agree with Tables II and III to the digits shown.

Check	What it tests	Result
Diatomic vs Eq. (9)	the general code in the solvable case	4.5×10^{-14}
Monatomic vs Eq. (8)	the $p = 1$ wrap-around branch	1.6×10^{-15}
Zone folding	equal masses in a $3a$ cell unfold onto Eq. (8) at matched q	3.5×10^{-14}
Sound velocity	$q \to 0$ slope against Eq. (7)	1.6×10^{-8}
$\omega_1(0) = 0$	translational invariance	0 exactly
Gap closure	equal masses leave no gap	pass
Per-bond springs	unequal bonds keep $\omega_1(0) = 0$ and the series-spring v_s	pass
Robustness	no `nan` or overflow; gaps unit-independent; $K < 0$ rejected	pass

TABLE I. Validation results. The first three are maximum absolute errors over the zone. The zone-folding check compares each frequency with the monatomic one at the corresponding unfolded wavevector, so it agrees to round-off. The last row groups regression checks for numerical edge cases: equal masses, extreme mass ratios and magnitudes, and unphysical input.

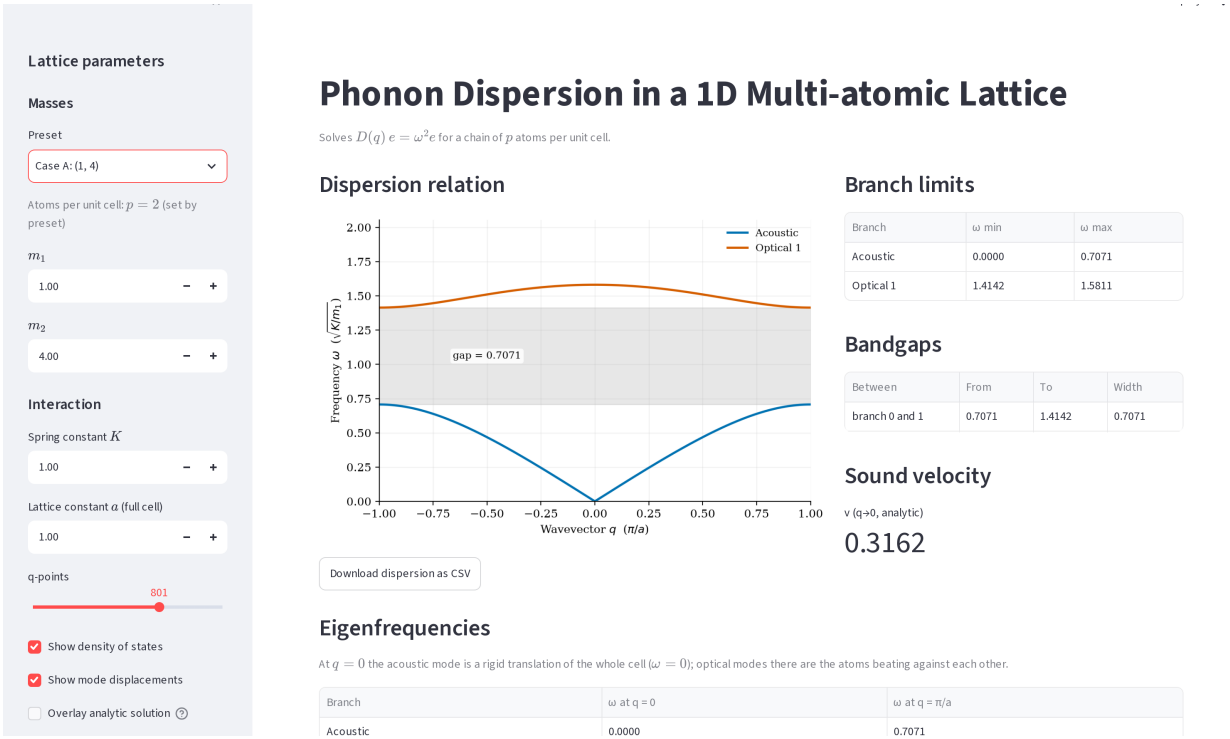


FIG. 3. The interface on Case A ($p = 2$). The sidebar carries the lattice parameters; the main panel gives the dispersion with the bandgap shaded and annotated, the per-branch frequency limits, every bandgap, the analytic sound velocity, and the zone-centre and zone-edge eigenfrequencies. Density-of-states and mode-displacement panels continue below the region shown.

V. RESULTS

All results use $K = 1$, $a = 1$ and quote frequencies in units of $\sqrt{K/m_1}$. Three mass/spring combinations are compared.

A. Case A — strong mass contrast

With $m_2/m_1 = 4$ the spectrum splits cleanly. The acoustic branch tops out at $\omega = 0.7071 = \sqrt{2K/m_2}$ and the optical branch bottoms at $\omega = 1.4142 = \sqrt{2K/m_1}$, both at the zone boundary and both matching the analytic zone-edge values exactly. The gap of 0.7071 is wide — it spans half the entire frequency range of the crystal. Physically, at the zone edge the two sublattices decouple: in the acoustic mode only the heavy atoms move appreciably, in the optical mode only the light ones, so each frequency is that of a single mass on its springs.

B. Case B — near-equal masses

Raising m_2 by only 10% above m_1 leaves a gap of 0.0658, barely a tenth of Case A's. The two branches very nearly join at the zone boundary, and the spectrum approaches the single folded branch of a monatomic chain. This is the diagnostic case: the gap is a consequence of

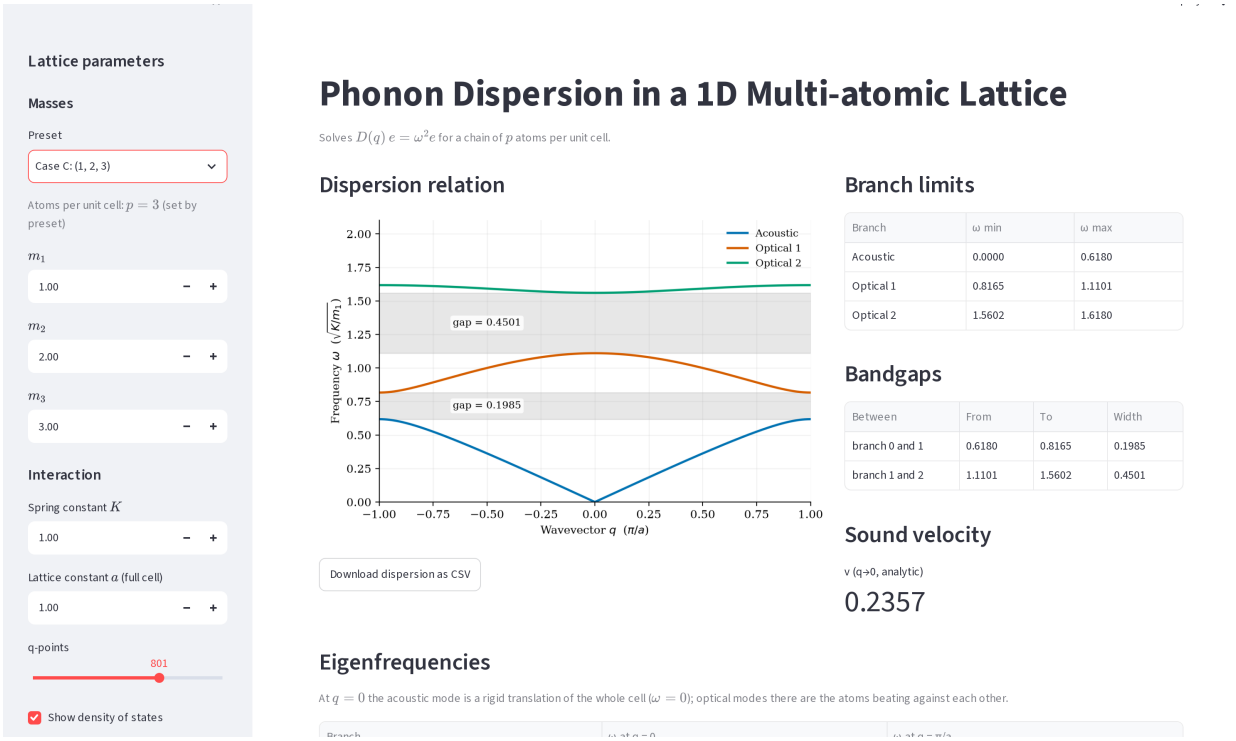


FIG. 4. The same interface on Case C ($p = 3$). Selecting the preset adds a third mass field, a third branch, and a second row to the bandgap table, with no change of code path. The displayed values match Tables II and III.

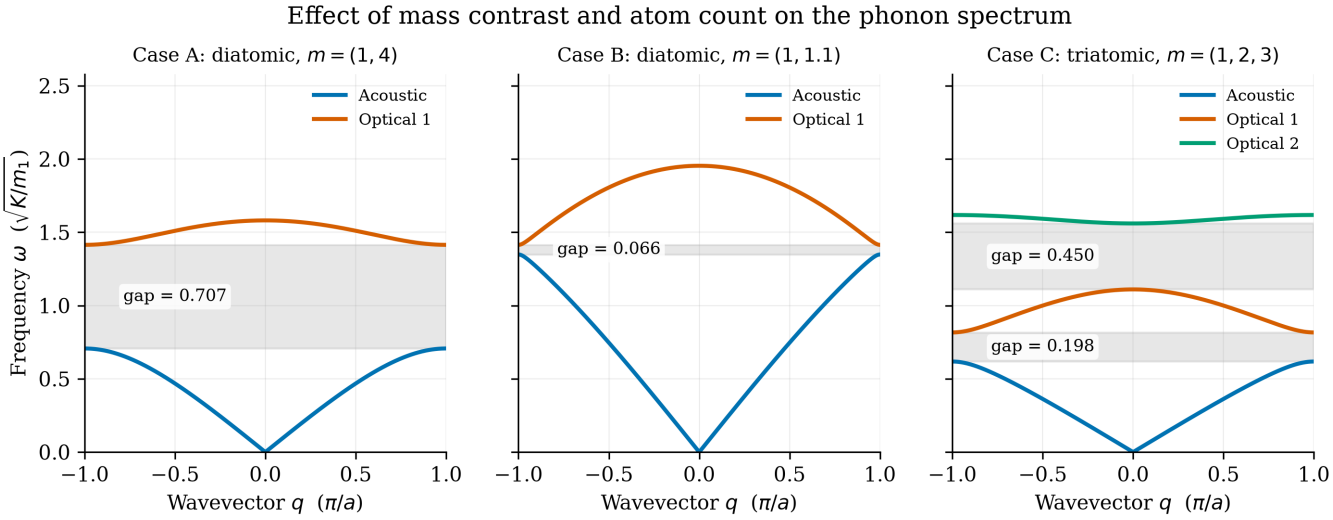


FIG. 5. Dispersion for the three cases on a common frequency scale. Shaded bands are forbidden frequency ranges. The acoustic branch starts linearly from the origin in all three.

the *two-member basis*, and it closes continuously as that basis becomes trivial, rather than being an independent property of the lattice. Figure 6 traces this directly.

C. Case C — three atoms per cell

With $p = 3$ there are one acoustic and two optical branches, and correspondingly two gaps, 0.1985 and 0.4501. The general rule is visible here: p atoms per cell give p branches and up to $p - 1$ gaps. The upper optical branch is noticeably flat, varying only between 1.5602

Case	Masses	Acoustic max	Optical range	Gap(s)	v_s
A	$(1, 4)$	0.7071	1.4142–1.5811	0.7071	0.3162
B	$(1, 1.1)$	1.3484	1.4142–1.9540	0.0658	0.4880
C	$(1, 2, 3)$	0.6180	0.8165–1.1101, 1.5602–1.6180	0.1985, 0.4501	0.2357

TABLE II. Computed branch limits, gaps and sound velocities, in units of $\sqrt{K/m_1}$.

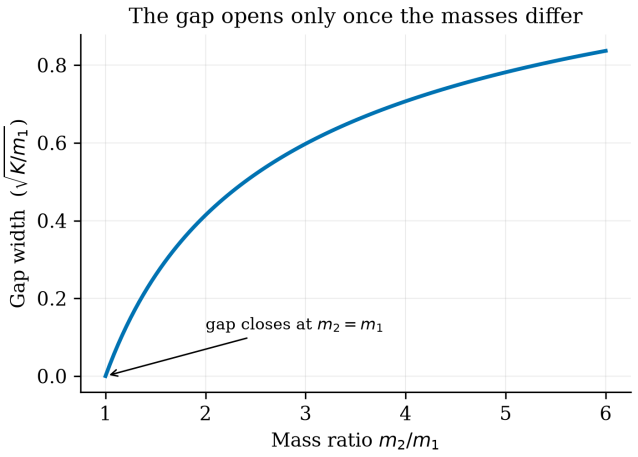


FIG. 6. Gap width against mass ratio for a diatomic chain. The gap grows monotonically from zero at $m_2 = m_1$; there is no threshold ratio below which a gap fails to appear.

and 1.6180 across the whole zone. A flat branch means a near-zero group velocity $\partial\omega/\partial q$: those modes carry almost no energy through the crystal, and they appear in the density of states as a sharp peak.

D. Effect of the spring constant

The spring constant enters (4)–(6) as an overall multiplier, so $D(q) \propto K$ and therefore $\omega \propto \sqrt{K}$ at every q. Figure 7 confirms this: rescaling by $\sqrt{K}$ collapses all four spectra onto one curve. Stiffer springs raise every frequency, including the gap width in absolute terms, but leave the *shape* of the spectrum — and hence all ratios, including gap-to-bandwidth — untouched. Mass contrast and spring constant therefore play different roles: masses set the structure of the spectrum, K sets only its scale.

E. Density of states

The density of states $g(\omega)$ counts modes per unit frequency, normalised per unit cell so that $\int g(\omega)\, d\omega = p$. It is zero inside every gap, which is what "forbidden frequency range" means in practice: an incident wave at such a frequency finds no mode to propagate in and is reflected. The sharp peaks are van Hove singularities, occurring where a branch has zero slope — at the zone centre and boundary, and across the whole of a flat branch. Case C's flat upper optical branch produces the most pronounced peak of the three, exactly as its shape in Figure 5 predicts. The bin-independent way to say this is through the bandwidth: that branch spans only 1.5602–1.6180, a width of 0.058, so its one state per cell is squeezed into a window about three times narrower than the next-narrowest branch (Case A's optical branch, width 0.167) and five times narrower than Case C's own middle branch.

F. Eigenfrequencies and their significance

Case	Branch	ω at $q = 0$	ω at $q = \pi/a$
A	Acoustic	0.0000	0.7071
	Optical 1	1.5811	1.4142
B	Acoustic	0.0000	1.3484
	Optical 1	1.9540	1.4142
C	Acoustic	0.0000	0.6180
	Optical 1	1.1101	0.8165
	Optical 2	1.5602	1.6180

TABLE III. Eigenfrequencies at the zone centre and zone boundary.

The eigenfrequencies are the natural frequencies of the lattice: a mode excited at $\omega_i(q)$ oscillates indefinitely at that frequency in the harmonic approximation, and any arbitrary motion of the chain decomposes into a superposition of them. Table III shows the two limiting cases.

At $q = 0$ all cells move identically. The acoustic eigenvalue is exactly zero in every case, since translating the whole crystal stretches no spring. The optical frequencies there are nonzero because the atoms inside a cell move against each other — for Case A, $1.5811 = \sqrt{2K(1/m_1 + 1/m_2)} = \sqrt{2K/\mu}$, with μ the reduced mass. Two masses on a single spring would give $\sqrt{K/\mu}$; the factor 2 appears because each atom sits between two springs, and in this mode both stretch by the same amount, so the restoring force is doubled. This is the mode that couples to infrared light in a real ionic crystal, and the origin of the name "optical".

At $q = \pi/a$ adjacent cells move in antiphase, the shortest wavelength the lattice supports. Both branches flatten here, which is why the gap edges in Table II are attained at the boundary. Figure 9 shows the eigenvectors directly, and makes the decoupling of Sec. V A explicit:

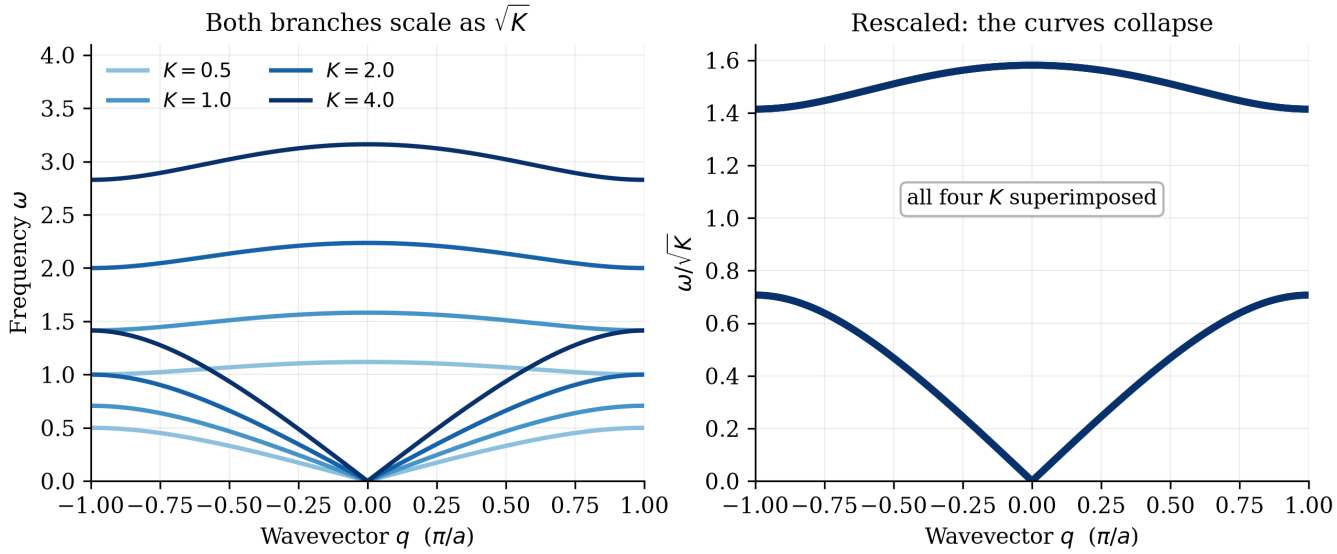


FIG. 7. Left: Case A dispersion at four spring constants. Right: the same curves divided by $\sqrt{K}$, collapsing onto a single pair.

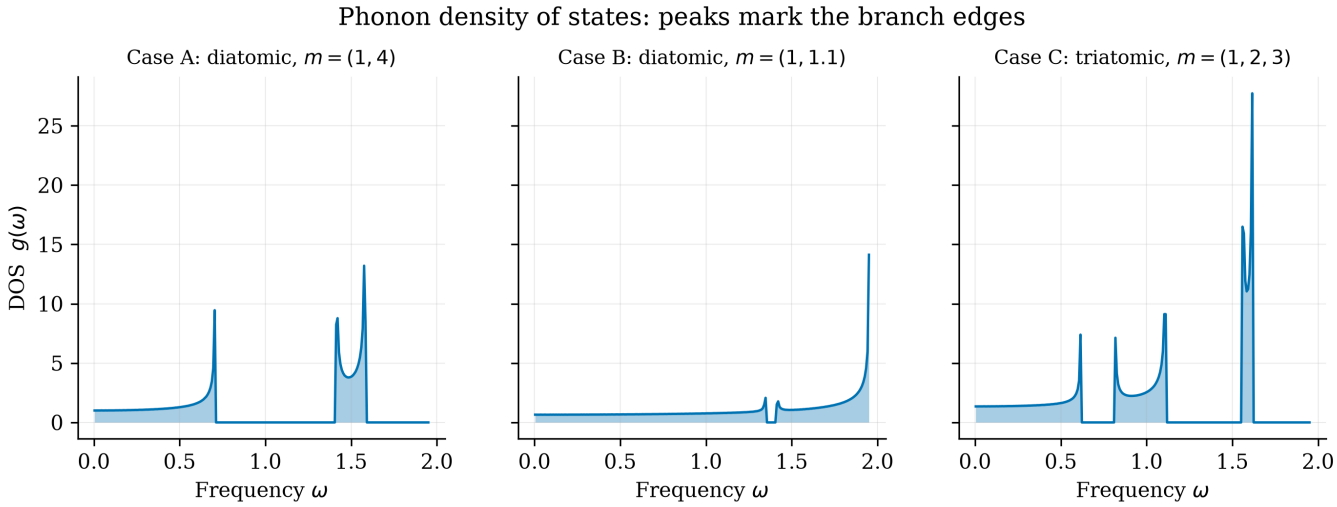


FIG. 8. Phonon density of states for the three cases, from a histogram of ω over a dense q mesh, normalised per unit cell so each panel integrates to its branch count p. Empty regions are the bandgaps of Table II. A histogram resolves the van Hove peaks but cannot represent the divergence itself: a $1/\sqrt{\omega - \omega_0}$ peak grows as the bins shrink. All three panels therefore use one common set of bins (width 8.1×10^{-3}), so that peak heights can be compared across panels, though not read as absolute values.

at the zone edge the acoustic eigenvector is $(0, 1)$ and the optical one $(1, 0)$, so each branch is a single sublattice vibrating while the other stands still. This needs $m_1 \neq m_2$. With equal masses the two zone-edge frequencies coincide, any combination of the two sublattice patterns is an equally valid mode, and the solver returns an arbitrary one; the interface detects that degeneracy and says so instead of presenting the drawn pattern as unique.

VI. CONCLUSIONS

Reducing the chain's equations of motion to a $p \times p$ Hermitian eigenvalue problem gives a method that handles any number of atoms per cell with no change of algorithm, and reproduces the textbook monatomic and diatomic solutions to within 10^{-13}. Applying it:

- p atoms per unit cell give one acoustic and $p - 1$ optical branches, with up to $p - 1$ gaps between them.

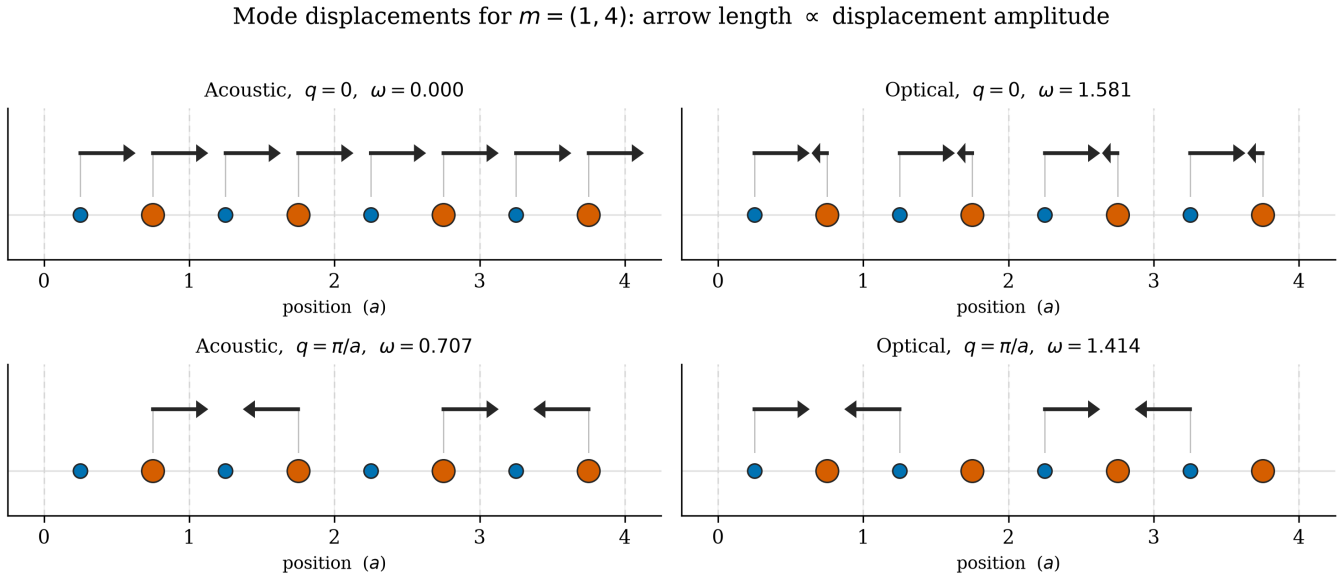


FIG. 9. Displacement patterns of the two branches of the $m = (1, 4)$ chain. Arrows show the *physical* displacement amplitudes $u_j = e_j/\sqrt{m_j}$, not the mass-weighted eigenvector components e_j returned by the solver; this is why the light atom swings four times as far as the heavy one in the optical mode at $q = 0$, keeping $\sum_j m_j u_j = 0$ so the cell's centre of mass stays at rest. Small circles are the light atoms, large circles the heavy ones. At $q = 0$ the acoustic mode translates every atom equally. At $q = \pi/a$ the two sublattices decouple completely — for $m_1 \neq m_2$ the eigenvectors are exactly $(0, 1)$ and $(1, 0)$ — so half the atoms carry no arrow at all, and the moving ones reverse from cell to cell through the factor $e^{iqna} = (-1)^n$.

- The bandgap is produced by mass contrast within the cell and closes continuously as the masses are equalised — 0.7071 at a mass ratio of 4, 0.0658 at a ratio of 1.1, zero at 1.
- The spring constant scales all frequencies as $\sqrt{K}$ and changes nothing else about the spectrum.
- Flat branches produce van Hove peaks in the density of states and carry negligible group velocity.

The obvious limitations are the harmonic approximation, which excludes thermal expansion and phonon lifetimes, and the restriction to nearest-neighbour coupling in one dimension.

REPRODUCING THIS WORK

```
uv venv --python 3.12 .venv
uv pip install numpy scipy matplotlib \
     streamlit pandas
cd question_3
python test_phonon.py    # checks
python make_figures.py   # figures
streamlit run app.py     # interface
```

Appendix A: Complete source code

All listings are included directly from the source files, so they cannot drift from the code that produced the results above. The files are, in dependency order: the physics core, its validation suite, the shared plotting style, the figure generator, and the interactive interface.

1. `phonon.py` — physics core

```
"""Phonon dispersion of a 1D lattice with p atoms per unit cell.

Convention: 'a' is the FULL unit-cell length, so the Brillouin zone is
q in [-pi/a, pi/a] and the diatomic case has cos(q*a/2) where texts that
set 'a' to the nearest-neighbour spacing write cos(q*a).

Spring constants: K is either one number (every bond equal) or a list of p
per-bond values, K[j] joining atom j to atom j+1 and K[p-1] joining the last
```

```
atom of a cell to the first atom of the next one.
"""

import numpy as np

EPS = np.finfo(float).eps


# ---- input handling ----

def _masses(m):
    m = np.atleast_1d(np.asarray(m, dtype=float))
    if m.ndim != 1 or m.size == 0:
        raise ValueError("masses must be a non-empty 1D list")
    if not np.all(np.isfinite(m)) or np.any(m <= 0):
        raise ValueError("masses must be finite and positive")
    return m


def _springs(K, p):
    """Per-bond spring constants as an array of length p."""
    K = np.atleast_1d(np.asarray(K, dtype=float))
    if K.size == 1:
        K = np.full(p, K[0])
    if K.size != p:
        raise ValueError(f"K must be a scalar or have one entry per bond ({p})")
    if not np.all(np.isfinite(K)) or np.any(K <= 0):
        # K <= 0 is not a lattice: the 'frequencies' would be imaginary
        raise ValueError("spring constants must be finite and positive")
    return K


# ---- dynamical matrix ----

def dynamical_matrix(q, m, K, a):
    """Mass-weighted dynamical matrix D(q) for masses m and springs K.

    Atom j couples to j+1 inside the cell; atom p couples back to atom 1 of
    the next cell, which is the only q-dependent term.
    """
    m = _masses(m)
    p = m.size
    K = _springs(K, p)
    sm = np.sqrt(m)            # sqrt(m_j)*sqrt(m_k), not sqrt(m_j*m_k): no overflow
    D = np.zeros((p, p), dtype=complex)

    if p == 1:
        # the single atom bonds to both neighbouring cells: 2K/m - 2K/m*cos(qa)
        D[0, 0] = 2.0 * K[0] / m[0] * (1.0 - np.cos(q * a))
        return D

    # each atom sits between the bond on its left and the one on its right
    np.fill_diagonal(D, (np.roll(K, 1) + K) / m)

    # intra-cell bonds
    for j in range(p - 1):
        c = -K[j] / (sm[j] * sm[j + 1])
        D[j, j + 1] = c
        D[j + 1, j] = c

    # inter-cell bond, carries all the q-dependence
    # += not =: at p=2 this bond shares the slot with the intra-cell one
    c = -K[-1] * np.exp(-1j * q * a) / (sm[0] * sm[-1])
    D[0, -1] += c
    D[-1, 0] += np.conj(c)
    return D


def _frequencies(w2, D):
    """sqrt of eigenvalues, clipping only round-off-sized negatives."""
    floor = -100.0 * EPS * max(np.abs(D).max(), 1e-300)
    if np.any(w2 < floor):
        raise ValueError("negative omega^2: the lattice is mechanically unstable")
    return np.sqrt(np.clip(w2, 0.0, None))


def frequencies_at(q, m, K=1.0, a=1.0):
    """Branch frequencies at a single wavevector, ascending."""
    D = dynamical_matrix(q, m, K, a)
    if not np.all(np.isfinite(D)):
        raise ValueError("dynamical matrix is not finite; inputs out of range")
    return _frequencies(np.linalg.eigvalsh(D), D)

```

```

def dispersion(m, K=1.0, a=1.0, nq=401, full_zone=True):
    """Frequencies omega(q) on a q grid. Returns (q, omega) with omega shape (nq, p).

    D(q) is Hermitian, so eigvalsh gives real, ascending eigenvalues: branch 0
    is acoustic, branches 1.. are optical. In a nearest-neighbour chain the
    branch extrema sit at q = 0 and q = pi/a, so the full-zone grid is forced
    to contain q = 0 (nq rounded up to odd); otherwise branch limits and gap
    edges would be read off points that miss the extrema.
    """
    m = _masses(m)
    qmax = np.pi / a
    if full_zone and nq % 2 == 0:
        nq += 1
    q = np.linspace(-qmax if full_zone else 0.0, qmax, nq)
    w = np.array([frequencies_at(qi, m, K, a) for qi in q])
    return q, w


# ---- derived quantities ----

def branch_limits(w):
    """(min, max) frequency of each branch."""
    return list(zip(w.min(axis=0), w.max(axis=0)))


def band_gaps(w, tol=1e-9):
    """Gaps between consecutive branches as (index, bottom, top, width).

    Branch i and i+1 are separated only if the whole of i lies below the whole
    of i+1 over the zone. `tol` is relative to the highest frequency, so the
    answer does not depend on the units of K and m.
    """
    thresh = tol * max(w.max(), 1e-300)
    gaps = []
    for i in range(w.shape[1] - 1):
        bottom, top = w[:, i].max(), w[:, i + 1].min()
        if top - bottom > thresh:
            gaps.append((i, bottom, top, top - bottom))
    return gaps


def sound_velocity(m, K=1.0, a=1.0):
    """Analytic acoustic slope at q->0.

    The p springs of a cell act in series, K_eff = 1/sum(1/K_j), and the whole
    cell mass moves: v = a*sqrt(K_eff/sum(m)). With equal springs this is
    a*sqrt(K/(p*sum(m))).
    """
    m = _masses(m)
    K = _springs(K, m.size)
    return a * np.sqrt(1.0 / np.sum(1.0 / K) / m.sum())


def is_folded(m, K=1.0, rtol=1e-9):
    """True if every mass and every spring in the cell is equal (p > 1).

    The true period is then a/p and the 'optical' branches are the acoustic
    branch folded back into the smaller zone, not genuine optical modes.
    """
    m = _masses(m)
    K = _springs(K, m.size)
    return m.size > 1 and np.ptp(m) <= rtol * m.max() and np.ptp(K) <= rtol * K.max()


def dos(m, K=1.0, a=1.0, nq=4001, nbins=200, bins=None):
    """Phonon DOS per unit cell by histogramming omega over a dense q mesh.

    Normalised so that the integral of g over all frequencies equals p, the
    number of branches: density=True gives unit area, so scale by p. The
    q = +pi/a endpoint is dropped because it is the same state as -pi/a.
    Pass explicit bin edges via `bins` to compare several lattices at one bin
    width; peak heights at a van Hove singularity depend on the bin width.
    """
    _, w = dispersion(m, K, a, nq=nq)
    w = w[:-1]
    counts, edges = np.histogram(w.ravel(), bins=nbins if bins is None else bins,
                                 density=True)
    # density=True normalises over the binned range; correct for any modes
    # that fall outside explicit edges (none do when edges span the spectrum)
    inside = np.mean((w.ravel() >= edges[0]) & (w.ravel() <= edges[-1]))
    return 0.5 * (edges[:-1] + edges[1:]), w.shape[1] * inside * counts

```

```python

# ---- closed forms, for validation ----

def monatomic_exact(q, m=1.0, K=1.0, a=1.0):
    """omega = 2*sqrt(K/m)*|sin(qa/2)|."""
    return 2.0 * np.sqrt(K / m) * np.abs(np.sin(q * a / 2.0))


def diatomic_exact(q, m1, m2, K=1.0, a=1.0):
    """Textbook two-branch solution. Returns (acoustic, optical).

    omega_-^2 is written as 4K sin^2/(m1 m2 (s + root)) rather than K(s - root),
    which avoids cancellation when the two terms are nearly equal; the square
    root argument is clipped because rounding can push it just below zero at
    the zone edge for equal masses.
    """
    s = 1.0 / m1 + 1.0 / m2
    sin2 = np.sin(q * a / 2.0) ** 2
    root = np.sqrt(np.clip(s**2 - 4.0 * sin2 / (m1 * m2), 0.0, None))
    w_minus2 = 4.0 * K * sin2 / (m1 * m2 * (s + root))
    return np.sqrt(w_minus2), np.sqrt(K * (s + root))


# ---- eigenvectors ----

def modes(q, m, K=1.0, a=1.0, rtol=1e-6):
    """Frequencies, real-space displacement patterns and degeneracy flags at one q.

    eigh returns the mass-weighted eigenvectors e; the physical displacement
    amplitude is u_j = e_j / sqrt(m_j), which is what gets drawn.

    `degenerate[b]` is True when branch b shares its frequency with another
    branch at this q. eigh then returns an arbitrary basis of the degenerate
    subspace, so the drawn pattern for that branch is not unique.
    """
    m = _masses(m)
    D = dynamical_matrix(q, m, K, a)
    w2, e = np.linalg.eigh(D)
    w = _frequencies(w2, D)
    u = e / np.sqrt(m)[:, None]

    # eigh fixes each column only up to a global phase, which at the zone edge
    # can be a pure i -- taking Re() then gives zero. Rotate so the largest
    # component of each mode is real and positive.
    lead = np.argmax(np.abs(u), axis=0)
    u = u * np.exp(-1j * np.angle(u[lead, np.arange(u.shape[1])]))
    u = u / np.abs(u).max(axis=0)  # normalise each mode for plotting

    tol = rtol * max(w2.max(), 1e-300)
    if w2.size == 1:
        return w, u, np.array([False])
    sep = np.abs(w2[:, None] - w2[None, :])
    np.fill_diagonal(sep, np.inf)          # a branch is not degenerate with itself
    return w, u, sep.min(axis=1) < tol
```

2. test_phonon.py — validation suite

```python
"""Validation checks from SCOPE.md. Run: python test_phonon.py"""

import numpy as np
import phonon as ph


def check(name, ok, detail=""):
    print(f"[{'PASS' if ok else 'FAIL'}] {name}" + (f"  {detail}" if detail else ""))
    return ok


# ---- 1. diatomic matches the closed form ----

def test_diatomic():
    m1, m2, K, a = 1.0, 4.0, 1.0, 1.0
    q, w = ph.dispersion([m1, m2], K, a, nq=401)
    ac, op = ph.diatomic_exact(q, m1, m2, K, a)
    err = max(np.abs(w[:, 0] - ac).max(), np.abs(w[:, 1] - op).max())
    return check("diatomic vs closed form", err < 1e-10, f"max err {err:.2e}")


# ---- 2. monatomic limit, including a folded cell of equal masses ----
```

```
23
24 def test_monatomic():
25     K, a = 1.0, 1.0
26     q, w = ph.dispersion([1.0], K, a, nq=401)
27     err = np.abs(w[:, 0] - ph.monatomic_exact(q, 1.0, K, a)).max()
28     ok = check("monatomic vs closed form", err < 1e-10, f"max err {err:.2e}")
29
30     # three equal masses in a cell of length 3a must be the monatomic branch
31     # folded into the smaller zone: at each q the three frequencies are the
32     # monatomic ones at q + 2*pi*n/(3a), n = -1, 0, 1. Compared at matched q,
33     # so this pins the frequency-to-wavevector map, not just the frequency set.
34     q3, w3 = ph.dispersion([1.0, 1.0, 1.0], K, 3 * a, nq=301)
35     folded = np.sort(np.stack([ph.monatomic_exact(q3 + 2 * np.pi * n / (3 * a), 1.0, K, a)
36                                for n in (-1, 0, 1)], axis=1), axis=1)
37     err = np.abs(w3 - folded).max()
38     return ok and check("equal-mass cell folds onto monatomic branch at matched q",
39                         err < 1e-10, f"max err {err:.2e}")
40
41
42 # ---- 3. acoustic slope at q->0 equals the analytic sound velocity ----
43
44 def test_sound_velocity():
45     worst = 0.0
46     for m in ([1.0, 4.0], [1.0, 2.0, 3.0], [2.0]):
47         a = 1.0
48         q, w = ph.dispersion(m, 1.0, a, nq=20001, full_zone=False)
49         n = 5  # slope from the first few points, where omega is still linear
50         v_num = np.polyfit(q[:n], w[:n, 0], 1)[0]
51         v_ana = ph.sound_velocity(m, 1.0, a)
52         worst = max(worst, abs(v_num - v_ana) / v_ana)
53     return check("acoustic sound velocity", worst < 1e-3, f"max rel err {worst:.2e}")
54
55
56 # ---- 4. acoustic branch vanishes at q=0 ----
57
58 def test_acoustic_zero():
59     worst = 0.0
60     for m in ([1.0, 4.0], [1.0, 2.0, 3.0], [1.0, 1.0, 2.0, 5.0]):
61         w = ph.frequencies_at(0.0, m, 1.0, 1.0)
62         worst = max(worst, w[0])
63     return check("omega_acoustic(q=0) = 0", worst < 1e-8, f"max {worst:.2e}")
64
65
66 # ---- 5. equal masses close the gap ----
67
68 def test_gap_closure():
69     _, w_eq = ph.dispersion([1.0, 1.0], nq=801)
70     _, w_un = ph.dispersion([1.0, 4.0], nq=801)
71     return check("equal masses -> no gap", len(ph.band_gaps(w_eq)) == 0) and \
72            check("unequal masses -> gap", len(ph.band_gaps(w_un)) == 1)
73
74
75 # ---- 6. per-bond springs: invariance and series sound velocity ----
76
77 def test_per_bond_springs():
78     m = [1.0, 2.0, 3.0]
79     _, w_scalar = ph.dispersion(m, 1.7, nq=201)
80     _, w_list = ph.dispersion(m, [1.7, 1.7, 1.7], nq=201)
81     ok = check("per-bond K list equal to scalar K", np.abs(w_scalar - w_list).max() < 1e-14)
82
83     # unequal bonds: omega_1(0) must still vanish and the slope must follow
84     # the series-spring formula K_eff = 1/sum(1/K_j)
85     K = [1.0, 3.0, 0.5]
86     # omega = sqrt(eigenvalue), so round-off of ~eps*|D| in the eigenvalue
87     # shows up as ~sqrt(eps*|D|) ~ 1e-8 in omega
88     w0 = ph.frequencies_at(0.0, m, K)[0]
89     q, w = ph.dispersion(m, K, nq=20001, full_zone=False)
90     v_num = np.polyfit(q[:5], w[:5, 0], 1)[0]
91     rel = abs(v_num - ph.sound_velocity(m, K)) / ph.sound_velocity(m, K)
92     return ok and check("unequal bonds: omega_1(0)=0 and series sound velocity",
93                         w0 < 1e-7 and rel < 1e-3, f"omega_1(0) {w0:.1e}, v rel err {rel:.1e}")
94
95
96 # ---- 7. robustness: units, round-off and unphysical input ----
97
98 def test_robustness():
99     # closed form stays finite for equal masses (rounding used to give NaN)
100    # and keeps full accuracy at extreme mass ratio (no cancellation)
101    q = np.linspace(-np.pi, np.pi, 2001)
102    ok = check("diatomic_exact finite for equal masses",
103               all(np.all(np.isfinite(ph.diatomic_exact(q, mm, mm)[0])) for mm in (0.09, 0.7, 1.0)))
104    qs = np.array([1e-4])
105    ac, _ = ph.diatomic_exact(qs, 1.0, 1e9)
```

```
    w_ref = qs * ph.sound_velocity([1.0, 1e9])
    ok &= check("diatomic_exact free of cancellation at mass ratio 1e9",
                abs(ac[0] - w_ref[0]) / w_ref[0] < 1e-6)

    # gap detection must not depend on the unit of K
    _, w_big = ph.dispersion([1.0, 1.0, 1.0], 1e14, nq=301)
    _, w_tiny = ph.dispersion([1.0, 4.0], 1e-20, nq=301)
    ok &= check("gap detection independent of units",
                len(ph.band_gaps(w_big)) == 0 and len(ph.band_gaps(w_tiny)) == 1)

    # huge masses no longer overflow the coupling into zero
    _, w_huge = ph.dispersion([1e160, 4e160], 1e160, nq=101)
    _, w_unit = ph.dispersion([1.0, 4.0], 1.0, nq=101)
    ok &= check("no overflow for masses ~1e160", np.allclose(w_huge, w_unit, rtol=1e-12))

    # an even nq must still put a grid point at q = 0
    _, w_even = ph.dispersion([1.0, 4.0], nq=400)
    ok &= check("even nq still samples q = 0", w_even[:, 0].min() == 0.0)

    # degenerate branches are flagged (their drawn pattern is an arbitrary mix)
    ok &= check("degeneracy flagged: equal masses at q = pi/a, not unequal",
                ph.modes(np.pi, [0.7, 0.7])[2].all() and not ph.modes(np.pi, [1.0, 4.0])[2].any())

    try:
        ph.dispersion([1.0, 4.0], -1.0)
        raised = False
    except ValueError:
        raised = True
    return ok and check("negative K rejected, not clipped to omega = 0", raised)


if __name__ == "__main__":
    results = [test_diatomic(), test_monatomic(), test_sound_velocity(),
               test_acoustic_zero(), test_gap_closure(), test_per_bond_springs(),
               test_robustness()]
    print(f"\n{sum(results)}/{len(results)} checks passed")
    raise SystemExit(0 if all(results) else 1)
```

3. common/style.py — shared plotting style

```
"""Shared matplotlib defaults, so Q3 and Q5 figures look like one document.

Palette is Okabe-Ito based and validated for colour-vision deficiency:
worst adjacent pair dE 11.0 (deutan). Branch identity is carried by colour
*and* by the legend plus direct labels, never by colour alone.
"""

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# categorical, assigned in this fixed order --- never cycled
ACOUSTIC = "#0072B2"
OPTICAL = ["#D55E00", "#009E73", "#9B59B6", "#56B4E9"]
GAP_FILL = "#B0B0B0"
EXACT = "#333333"


def branch_colors(p):
    """Acoustic first, then optical. Wraps into greys past 5 branches."""
    return [ACOUSTIC] + [OPTICAL[i % len(OPTICAL)] for i in range(p - 1)]


def apply():
    """Serif to match the LaTeX body text; grid and spines kept recessive."""
    plt.rcParams.update({
        "font.family": "serif",
        "font.size": 10,
        "axes.titlesize": 11,
        "axes.labelsize": 10,
        "axes.linewidth": 0.8,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": True,
        "grid.alpha": 0.25,
        "grid.linewidth": 0.5,
        "lines.linewidth": 2.0,
        "legend.frameon": False,
        "legend.fontsize": 9,
```

```
        "figure.dpi": 150,
        "savefig.dpi": 300,
        "savefig.bbox": "tight",
    })
```

4. make_figures.py — figure generation

```
"""Regenerates every figure in figures/. Run: python make_figures.py"""

import os
import sys

import numpy as np
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "common"))
import style
import phonon as ph
import _schematic

FIG = os.path.join(os.path.dirname(__file__), "figures")

# the three required combinations, plus labels used in the report
CASES = [
    ("case_a", [1.0, 4.0], 1.0, "Case A: diatomic, $m=(1,4)$"),
    ("case_b", [1.0, 1.1], 1.0, "Case B: diatomic, $m=(1,1.1)$"),
    ("case_c", [1.0, 2.0, 3.0], 1.0, "Case C: triatomic, $m=(1,2,3)$"),
]


# ---- dispersion with gaps shaded ----

def plot_dispersion(ax, m, K, a=1.0, nq=801, annotate_gaps=True, ymax=None):
    q, w = ph.dispersion(m, K, a, nq=nq)
    colors = style.branch_colors(len(m))

    # gap labels sit left of centre; the legend takes the upper right
    for gap in ph.band_gaps(w):
        _, bottom, top, width = gap
        ax.axhspan(bottom, top, color=style.GAP_FILL, alpha=0.3, zorder=0)
        if annotate_gaps:
            ax.annotate(f"gap = {width:.3f}", xy=(-0.5, 0.5 * (bottom + top)),
                        ha="center", va="center", fontsize=8.5,
                        bbox=dict(boxstyle="round,pad=0.25", fc="white", ec="none", alpha=0.85))

    for i in range(w.shape[1]):
        label = "Acoustic" if i == 0 else f"Optical {i}"
        ax.plot(q / np.pi, w[:, i], color=colors[i], label=label)

    ax.set_xlabel(r"Wavevector $q$  ($\pi/a$)")
    ax.set_ylabel(r"Frequency $\omega$  ($\sqrt{K/m_1}$)")
    ax.set_xlim(-1, 1)
    # headroom so the legend never lands on a curve
    ax.set_ylim(0, (ymax if ymax else w.max()) * 1.32)
    return q, w


def _common_ymax():
    return max(ph.dispersion(m, K, nq=401)[1].max() for _, m, K, _ in CASES)


def fig_cases():
    for name, m, K, title in CASES:
        fig, ax = plt.subplots(figsize=(5.2, 3.6))
        plot_dispersion(ax, m, K)
        ax.set_title(title)
        ax.legend(loc="upper right")
        fig.savefig(f"{FIG}/{name}.png")
        plt.close(fig)


def fig_comparison():
    """All three cases side by side --- the gap is the thing that changes."""
    fig, axes = plt.subplots(1, 3, figsize=(11, 3.4), sharey=True)
    ymax = _common_ymax()
    for ax, (_, m, K, title) in zip(axes, CASES):
        plot_dispersion(ax, m, K, ymax=ymax)
        ax.set_title(title, fontsize=9.5)
        ax.legend(loc="upper right", fontsize=8)
```

```python
    for ax in axes[1:]:
        ax.set_ylabel("")
    fig.suptitle("Effect of mass contrast and atom count on the phonon spectrum", y=1.02)
    fig.savefig(f"{FIG}/comparison.png")
    plt.close(fig)


# ---- spring-constant sweep: omega scales as sqrt(K), gap ratios do not move ----

def fig_spring_sweep():
    m = [1.0, 4.0]
    Ks = [0.5, 1.0, 2.0, 4.0]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.6))

    # K is ordered, so a single-hue light->dark ramp, not a categorical set
    greys = plt.cm.Blues(np.linspace(0.42, 1.0, len(Ks)))
    for K, c in zip(Ks, greys):
        q, w = ph.dispersion(m, K, nq=601)
        ax1.plot(q / np.pi, w[:, 0], color=c, label=f"$K={K}$")
        ax1.plot(q / np.pi, w[:, 1], color=c)
    ax1.set_xlabel(r"Wavevector $q$  ($\pi/a$)")
    ax1.set_ylabel(r"Frequency $\omega$")
    ax1.set_xlim(-1, 1)
    ax1.set_ylim(0, 4.1)  # headroom so the legend clears the K=4 curve
    ax1.set_title("Both branches scale as $\\sqrt{K}$")
    ax1.legend(loc="upper left", fontsize=9, ncol=2)

    # rescaling by sqrt(K) collapses every curve onto one
    for K, c in zip(Ks, greys):
        q, w = ph.dispersion(m, K, nq=601)
        ax2.plot(q / np.pi, w[:, 0] / np.sqrt(K), color=c, lw=3.2)
        ax2.plot(q / np.pi, w[:, 1] / np.sqrt(K), color=c, lw=3.2)
    ax2.set_xlabel(r"Wavevector $q$  ($\pi/a$)")
    ax2.set_ylabel(r"$\omega/\sqrt{K}$")
    ax2.set_xlim(-1, 1)
    ax2.set_ylim(bottom=0)
    ax2.set_title("Rescaled: the curves collapse")
    # no legend here: all four curves lie on top of each other, which is the point
    ax2.annotate("all four $K$ superimposed", xy=(0.0, 1.05), ha="center", fontsize=9.5,
                 bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="0.7"))

    fig.savefig(f"{FIG}/spring_sweep.png")
    plt.close(fig)


# ---- gap width vs mass ratio ----

def fig_gap_vs_ratio():
    ratios = np.linspace(1.0, 6.0, 120)
    widths = []
    for r in ratios:
        _, w = ph.dispersion([1.0, r], 1.0, nq=401)
        gaps = ph.band_gaps(w)
        widths.append(gaps[0][3] if gaps else 0.0)

    fig, ax = plt.subplots(figsize=(5.2, 3.4))
    ax.plot(ratios, widths, color=style.ACOUSTIC)
    ax.set_xlabel(r"Mass ratio $m_2/m_1$")
    ax.set_ylabel(r"Gap width  ($\sqrt{K/m_1}$)")
    ax.set_title("The gap opens only once the masses differ")
    ax.annotate("gap closes at $m_2=m_1$", xy=(1.0, 0.0), xytext=(2.0, 0.12),
                fontsize=8.5, arrowprops=dict(arrowstyle="->", lw=0.8))
    fig.savefig(f"{FIG}/gap_vs_ratio.png")
    plt.close(fig)


# ---- validation: numerics on top of the closed form ----

def fig_validation():
    m1, m2 = 1.0, 4.0
    q, w = ph.dispersion([m1, m2], 1.0, nq=201)
    ac, op = ph.diatomic_exact(q, m1, m2)

    fig, ax = plt.subplots(figsize=(5.2, 3.6))
    ax.plot(q / np.pi, ac, color=style.EXACT, lw=3.0, alpha=0.35, label="Analytic")
    ax.plot(q / np.pi, op, color=style.EXACT, lw=3.0, alpha=0.35)
    ax.plot(q[::4] / np.pi, w[::4, 0], ls="none", marker="o", ms=4,
            color=style.ACOUSTIC, label="Numerical (acoustic)")
    ax.plot(q[::4] / np.pi, w[::4, 1], ls="none", marker="s", ms=4,
            color=style.OPTICAL[0], label="Numerical (optical)")
    ax.set_xlabel(r"Wavevector $q$  ($\pi/a$)")
    ax.set_ylabel(r"Frequency $\omega$  ($\sqrt{K/m_1}$)")
    ax.set_xlim(-1, 1)
```

```
    ax.set_ylim(bottom=0)
    ax.set_title("Diatomic chain: eigensolver vs closed form")
    ax.legend(loc="center")
    fig.savefig(f"{FIG}/validation.png")
    plt.close(fig)


# ---- density of states ----

def fig_dos():
    fig, axes = plt.subplots(1, 3, figsize=(11, 3.2), sharey=True)
    # one set of bin edges for all three cases, so every panel has the same bin
    # width: a van Hove peak's height grows as the bin shrinks, so heights are
    # only comparable across panels at a common width
    edges = np.linspace(0.0, _common_ymax() * 1.0001, 241)
    for ax, (_, m, K, title) in zip(axes, CASES):
        centres, g = ph.dos(m, K, nq=40001, bins=edges)
        ax.fill_between(centres, g, color=style.ACOUSTIC, alpha=0.35, lw=0)
        ax.plot(centres, g, color=style.ACOUSTIC, lw=1.2)
        ax.set_xlabel(r"Frequency $\omega$")
        ax.set_title(title, fontsize=9.5)
    axes[0].set_ylabel(r"DOS  $g(\omega)$")
    fig.suptitle("Phonon density of states: peaks mark the branch edges", y=1.03)
    fig.savefig(f"{FIG}/dos.png")
    plt.close(fig)


# ---- lattice schematic ----

def fig_schematic():
    _schematic.make(f"{FIG}/schematic.png")


# ---- mode displacements ----

def fig_modes():
    """Case A at the zone centre and edge: what each branch actually does."""
    m, K, a, ncell = [1.0, 4.0], 1.0, 1.0, 4
    p = len(m)
    scale = 0.36          # displacement -> data units
    y_atom, y_arrow = 0.0, 0.30

    fig, axes = plt.subplots(2, 2, figsize=(10, 4.2))
    for row, (q_frac, qlab) in enumerate([(0.0, r"$q=0$"), (1.0, r"$q=\pi/a$")]):
        q = q_frac * np.pi / a
        w, u, _ = ph.modes(q, m, K, a)
        for col, bname in enumerate(["Acoustic", "Optical"]):
            ax = axes[row, col]
            for n in range(ncell):
                for j in range(p):
                    x = n * a + (j + 0.5) * a / p
                    d = np.real(u[j, col] * np.exp(1j * q * n * a))
                    # atoms always drawn, sized by mass; stationary ones keep their marker
                    ax.plot([x], [y_atom], marker="o",
                            ms=7.5 * (m[j] / m[0]) ** (1 / 3),
                            color=_schematic.ATOM_COLORS[j],
                            markeredgecolor="0.15", markeredgewidth=0.8, zorder=3)
                    if abs(d) > 1e-9:
                        # arrows on their own row so no amplitude hides behind a marker
                        # head length scales with the arrow, so a short arrow is not
                        # swallowed by a fixed-size head -- the 4:1 ratio must stay readable
                        L = scale * d
                        ax.arrow(x, y_arrow, L, 0, length_includes_head=True,
                                 head_width=0.052, head_length=min(0.05, 0.34 * abs(L)),
                                 lw=1.9, fc="0.15", ec="0.15", zorder=4)
                        ax.plot([x, x], [y_atom + 0.09, y_arrow], color="0.75", lw=0.7,
                                zorder=1)
            ax.axhline(y_atom, color="0.88", lw=0.8, zorder=0)
            for n in range(ncell + 1):
                ax.axvline(n * a, color="0.88", lw=0.8, linestyle=(0, (4, 3)), zorder=0)
            ax.set_title(f"{bname},  {qlab},  " + rf"$\omega={w[col]:.3f}$", fontsize=10)
            ax.set_xlim(-0.25, ncell * a + 0.25)
            ax.set_ylim(-0.22, 0.52)
            ax.set_yticks([])
            ax.set_xticks(range(ncell + 1))
            ax.set_xlabel("position  ($a$)", fontsize=9)

    fig.suptitle(r"Mode displacements for $m=(1,4)$: arrow length $\propto$ "
                 r"displacement amplitude", y=1.02)
    fig.tight_layout()
    fig.savefig(f"{FIG}/modes.png")
    plt.close(fig)

```

```
239
240 if __name__ == "__main__":
241     style.apply()
242     os.makedirs(FIG, exist_ok=True)
243     fig_schematic()
244     fig_cases()
245     fig_modes()
246     fig_comparison()
247     fig_spring_sweep()
248     fig_gap_vs_ratio()
249     fig_validation()
250     fig_dos()
251     print("figures written to", os.path.abspath(FIG))
```

5. app.py — interactive interface

```
1  """Streamlit interface for 1D multi-atomic phonon dispersion.
2
3  Run:  streamlit run app.py
4  """
5
6  import os
7  import sys
8
9  import numpy as np
10 import pandas as pd
11 import streamlit as st
12 from matplotlib.figure import Figure
13
14 _HERE = os.path.dirname(os.path.abspath(__file__))
15 # when run as a page of streamlit_app.py the entry script's directory is on
16 # sys.path, not this one, so add both explicitly. Streamlit re-runs this script
17 # on every widget change, so insert only once or the list grows without bound.
18 for _p in (os.path.normpath(os.path.join(_HERE, "..", "common")), _HERE):
19     if _p not in sys.path:
20         sys.path.insert(0, _p)
21 import style
22 import phonon as ph
23
24 style.apply()
25 # only the entry script may configure the page; harmless no-op when this
26 # module runs as a page of streamlit_app.py
27 try:
28     st.set_page_config(page_title="Phonon Dispersion --- 1D Multi-atomic Lattice", layout="wide")
29 except Exception:
30     pass
31 st.title("Phonon Dispersion in a 1D Multi-atomic Lattice")
32 st.caption(r"Solves $D(q)\,e = \omega^2 e$ for a chain of $p$ atoms per unit cell.")
33
34 # ---- inputs ----
35
36 with st.sidebar:
37     st.header("Lattice parameters")
38     st.subheader("Masses")
39     preset = st.selectbox("Preset", ["Custom", "Case A: (1, 4)", "Case B: (1, 1.1)",
40                                      "Case C: (1, 2, 3)"])
41     defaults = {"Case A: (1, 4)": [1.0, 4.0],
42                 "Case B: (1, 1.1)": [1.0, 1.1],
43                 "Case C: (1, 2, 3)": [1.0, 2.0, 3.0]}
44
45     # a preset fixes p, so show it rather than a slider that would disagree with it
46     if preset in defaults:
47         base = defaults[preset]
48         p = len(base)
49         st.caption(f"Atoms per unit cell: $p = {p}$  (set by preset)")
50     else:
51         p = st.slider("Number of atoms per unit cell, $p$", 1, 5, 2)
52         base = [1.0 + i for i in range(p)]
53
54     # the key carries the preset: a keyed widget ignores `value` once it exists,
55     # so without this, switching preset would leave the old masses in place
56     # upper bounds keep D(q) far from floating-point overflow
57     masses = [st.number_input(f"$m_{{{j+1}}}$", min_value=0.01, max_value=1e4,
58                               value=float(base[j]), step=0.1, key=f"m_{preset}_{j}")
59               for j in range(p)]
60
61     st.subheader("Interaction")
62     K = st.number_input("Spring constant $K$", min_value=0.01, max_value=1e4, value=1.0,
63                         step=0.1)
```

```python
    a = st.number_input("Lattice constant $a$ (full cell)", min_value=0.01, max_value=1e3,
                        value=1.0, step=0.1)
    nq = st.select_slider("q-points", [201, 401, 801, 1601], value=801)
    show_dos = st.checkbox("Show density of states", value=True)
    show_modes = st.checkbox("Show mode displacements", value=True)
    show_exact = st.checkbox("Overlay analytic solution", value=False,
                             help="Available for p = 1 and p = 2.")

# ---- solve ----


@st.cache_data(max_entries=64)
def solve(masses, K, a, nq):
    return ph.dispersion(list(masses), K, a, nq=nq)


try:
    q, w = solve(tuple(masses), K, a, nq)
    if not np.all(np.isfinite(w)):
        raise ValueError("non-finite frequencies")
except ValueError as err:
    st.error(f"These inputs do not give a stable, finite lattice ({err}).")
    st.stop()

gaps = ph.band_gaps(w)
limits = ph.branch_limits(w)
colors = style.branch_colors(p)
# equal masses: the real period is a/p, so the upper branches are the acoustic
# branch folded into the smaller zone, not optical modes
folded = ph.is_folded(masses, K)
names = ["Acoustic"] + [("Folded acoustic " if folded else "Optical ") + str(i)
                        for i in range(1, p)]

# ---- dispersion plot ----

left, right = st.columns([3, 2])

with left:
    st.subheader("Dispersion relation")
    fig = Figure(figsize=(7, 4.6))
    ax = fig.subplots()

    for _, bottom, top, width in gaps:
        ax.axhspan(bottom, top, color=style.GAP_FILL, alpha=0.3, zorder=0)
        ax.annotate(f"gap = {width:.4f}", xy=(-0.5, 0.5 * (bottom + top)),
                    ha="center", va="center", fontsize=9,
                    bbox=dict(boxstyle="round,pad=0.25", fc="white", ec="none", alpha=0.85))

    for i in range(p):
        ax.plot(q / np.pi, w[:, i], color=colors[i], label=names[i])

    if show_exact and p <= 2:
        if p == 1:
            ax.plot(q / np.pi, ph.monatomic_exact(q, masses[0], K, a),
                    color=style.EXACT, lw=3.0, alpha=0.3, zorder=0, label="Analytic")
        else:
            ac, op = ph.diatomic_exact(q, masses[0], masses[1], K, a)
            ax.plot(q / np.pi, ac, color=style.EXACT, lw=3.0, alpha=0.3, zorder=0,
                    label="Analytic")
            ax.plot(q / np.pi, op, color=style.EXACT, lw=3.0, alpha=0.3, zorder=0)
    elif show_exact:
        st.caption("No closed form for $p > 2$; the overlay is shown only for $p \\le 2$.")

    ax.set_xlabel(r"Wavevector $q$  ($\pi/a$)")
    ax.set_ylabel(r"Frequency $\omega$  (input units)")
    ax.set_xlim(-1, 1)
    ax.set_ylim(0, w.max() * 1.3)
    ax.legend(loc="upper right")
    st.pyplot(fig)
    st.caption(r"Frequencies are in the units set by the inputs, $\omega\propto\sqrt{K/m}$; "
               r"they are in units of $\sqrt{K/m_1}$ only when $K = m_1 = 1$.")
    if folded:
        st.info(f"{'Both' if p == 2 else f'All {p}'} masses are equal, so the true period is $a/{p}$ and the upper "
                "branches are the acoustic branch folded back into this zone, not "
                "optical modes.")

    csv = "q," + ",".join(["acoustic"] + [f"optical_{i}" for i in range(1, p)]) + "\n"
    csv += "\n".join(",".join(f"{v:.6g}" for v in row) for row in np.column_stack([q, w]))
    st.download_button("Download dispersion as CSV", csv, "dispersion.csv", "text/csv")

with right:
    st.subheader("Branch limits")
    st.dataframe(pd.DataFrame({
```

```
        "Branch": names,
        "ω min": [f"{lo:.4f}" for lo, _ in limits],
        "ω max": [f"{hi:.4f}" for _, hi in limits],
    }), hide_index=True, width="stretch")

    st.subheader("Bandgaps")
    if gaps:
        st.dataframe(pd.DataFrame({
            "Between": [f"branch {i} and {i+1}" for i, _, _, _ in gaps],
            "From": [f"{b:.4f}" for _, b, _, _ in gaps],
            "To": [f"{t:.4f}" for _, _, t, _ in gaps],
            "Width": [f"{wd:.4f}" for _, _, _, wd in gaps],
        }), hide_index=True, width="stretch")
    elif p == 1:
        st.info("One atom per cell gives a single branch, so there is no gap to open.")
    else:
        st.info("No gap: the branches touch. Equal masses leave the spectrum unsplit.")

    st.subheader("Sound velocity")
    st.metric("v (q→0, analytic)", f"{ph.sound_velocity(masses, K, a):.4f}")

# ---- eigenfrequencies at the zone centre and edge ----

st.subheader("Eigenfrequencies")
st.caption("At $q=0$ the acoustic mode is a rigid translation of the whole cell "
           "($\\omega=0$); optical modes there are the atoms beating against each other.")

w_centre = ph.frequencies_at(0.0, masses, K, a)
w_edge = ph.frequencies_at(np.pi / a, masses, K, a)
st.dataframe(pd.DataFrame({
    "Branch": names,
    "ω at q = 0": [f"{v:.4f}" for v in w_centre],
    "ω at q = π/a": [f"{v:.4f}" for v in w_edge],
}), hide_index=True, width="stretch")

# ---- density of states ----

if show_dos:
    st.subheader("Density of states")
    centres, g = ph.dos(masses, K, a, nbins=160)
    fig = Figure(figsize=(7, 2.8))
    ax = fig.subplots()
    ax.fill_between(centres, g, color=style.ACOUSTIC, alpha=0.35, lw=0)
    ax.plot(centres, g, color=style.ACOUSTIC, lw=1.2)
    ax.set_xlabel(r"Frequency $\omega$")
    ax.set_ylabel(r"$g(\omega)$")
    st.pyplot(fig)
    st.caption("Normalised per unit cell, so the area under the curve is the number of "
               "branches p. Peaks are van Hove singularities, where a branch flattens "
               "and many q-states share one frequency. Peak heights depend on the bin "
               "width, so read peak positions, not heights.")

# ---- mode displacements ----

if show_modes:
    st.subheader("Mode displacements")
    st.caption("Arrows show the instantaneous displacement of each atom. "
               "Pick a branch and a wavevector to see how that mode actually moves.")

    c1, c2 = st.columns(2)
    with c1:
        branch = st.selectbox("Branch", names)
    with c2:
        q_frac = st.slider("q  (units of π/a)", 0.0, 1.0, 0.0, 0.05)

    b = names.index(branch)
    q_sel = q_frac * np.pi / a
    w_sel, u_sel, degen = ph.modes(q_sel, masses, K, a)

    ncells = 4
    y_atom, y_arrow = 0.0, 0.30 * a      # arrows sit on their own row above the chain
    atom_colors = [style.ACOUSTIC] + style.OPTICAL

    # real part of u_j * exp(iqna): the displacement at this instant. Collected
    # first so the arrow scale can be set from the largest one.
    disp = np.array([[np.real(u_sel[j, b] * np.exp(1j * q_sel * n * a))
                      for j in range(p)] for n in range(ncells)])
    dmax = np.abs(disp).max()
    # longest arrow reaches 70% of the atom spacing, whatever p and however u is normalised
    scale = 0.7 * (a / p) / dmax if dmax > 1e-9 else 0.0
    mmin = min(masses)

    fig = Figure(figsize=(8, 2.6))
```

```
    ax = fig.subplots()
    for n in range(ncells):
        for j in range(p):
            x = n * a + (j + 0.5) * a / p
            d = disp[n, j]
            # atoms always drawn, sized by mass, so stationary ones stay visible
            ax.plot([x], [y_atom], marker="o",
                    ms=min(16.0, 7.5 * (masses[j] / mmin) ** (1 / 3)),
                    color=atom_colors[j % len(atom_colors)],
                    markeredgecolor="0.15", markeredgewidth=0.8, zorder=3)
            if abs(d) > 1e-9:
                # head length scales with the arrow, so a short arrow is not
                # swallowed by a fixed-size head and amplitude ratios stay readable
                L = scale * d
                ax.arrow(x, y_arrow, L, 0, length_includes_head=True,
                         head_width=0.052 * a, head_length=min(0.05 * a, 0.34 * abs(L)),
                         lw=1.9, fc="0.15", ec="0.15", zorder=4)
                ax.plot([x, x], [y_atom + 0.09 * a, y_arrow], color="0.75", lw=0.7,
                        zorder=1)
    ax.axhline(y_atom, color="0.88", lw=0.8, zorder=0)
    for n in range(ncells + 1):
        ax.axvline(n * a, color="0.88", lw=0.8, linestyle=(0, (4, 3)), zorder=0)
    ax.set_xlim(-0.25 * a, (ncells + 0.25) * a)
    ax.set_ylim(-0.22 * a, 0.52 * a)
    ax.set_yticks([])
    ax.set_xticks([n * a for n in range(ncells + 1)])
    ax.set_xticklabels(range(ncells + 1))
    ax.set_xlabel("Position along the chain  ($a$)")
    ax.set_title(f"{branch} mode at $q = {q_frac:.2f}\\,\\pi/a$,  "
                 f"$\\omega = {w_sel[b]:.4f}$")
    st.pyplot(fig)

    if degen[b]:
        st.warning("This branch is degenerate with another one at this $q$. Any mixture "
                   "of the two is an equally valid mode, so the pattern drawn is one "
                   "arbitrary choice, not a unique displacement.")
    elif b == 0 and q_frac == 0.0:
        st.info("At $q=0$ the acoustic mode is a uniform translation --- every atom moves "
                "together, no spring stretches, so $\\omega = 0$.")
    elif b > 0 and q_frac == 0.0 and not folded:
        st.info("At $q=0$ an optical mode has the atoms inside each cell moving against "
                "one another, with the cell's centre of mass at rest.")
```


[1] C. Kittel, *Introduction to Solid State Physics*, 8th ed., Wiley, 2005. Chapter 4, lattice vibrations and the diatomic chain.

[2] N. W. Ashcroft and N. D. Mermin, *Solid State Physics*, Holt–Saunders, 1976. Chapters 22–23, the harmonic crystal.

[3] M. T. Dove, *Introduction to Lattice Dynamics*, Cambridge University Press, 1993.

[4] D. Tong, *Lectures on Solid State Physics*, University of Cambridge. https://www.damtp.cam.ac.uk/user/tong/aqm.html

[5] C. R. Harris *et al.*, "Array programming with NumPy", *Nature* **585**, 357–362 (2020). `numpy.linalg.eigvalsh` documentation.

[6] J. D. Hunter, "Matplotlib: A 2D graphics environment", *Computing in Science & Engineering* **9**, 90–95 (2007).

[7] Streamlit documentation. https://docs.streamlit.io